UNIT-1

→ Toc terminology

(1) Symbol (2) Alphabet (3) String

(4) Language.

(i) Symbol:- collection of characters is called symbol

→ symbols are infinite.

Eg:- a, b, c ... z, A to Z, 0 to 9, $, ?, etc

(ii) Alphabet:- collection of symbols is called Alphabet.

→ It is represented by Σ

i.e $\Sigma = \{0, 1\}$, $\Sigma = \{a, b, c\}$ etc

→ Alphabet is a finite set.

(iii) String:- collection of sequence of symbols is called String.

Eg:- a, aa, abcd, baba ... etc

(iv) Language:- It is nothing but set of strings is called language. (or) set of strings on over alphabet is called language.

Eg:

Eg1:- L_1 = set of all strings of length 2 on over $\Sigma = \{0, 1\}$.

$= \{00, 01, 10, 11\}$

L_2: set of all strings of length 1 over $\{0,1\}$

$= \{0, 1\}$

L_3: set of all string of length 0:

→ The length "0" is represented by using Epsilon i.e ϵ

$|\epsilon| = 0, \quad |abc| = 3$

L_4: set of all string start with 0 on over alphabet $\{0,1\}$

$= \{0, 01, 000, 011, 01010 \ldots$ etc $\}$

→ L_1, L_2 & L_3 are finite languages

→ L_4 is infinite Languages.

L_5: set of all strings that Contains atleast "a" on over $\Sigma = \{a, b\}$

i.e The string must Contain "a"

$= \{a, ba, aaa, bbb \ldots$ etc $\}$

L_5 is also infinite language.

L_6: set of all strings that end with b on $\Sigma = \{a, b\}$

$= \{b, ab, bbb, abbb, bbba \ldots\}$

Powel of alphabet (Σ):-

→ Let $\Sigma = \{a, b\}$

Σ^1 = set of all strings of length 1 = $\{a, b\}$

Σ^2 = set of all strings of length 2 = $\{aa, ba, bb, ab\}$

Σ^3 = set of all strings of length 3 =

Σ^0 = set of all strings length '0' = $\{\epsilon\}$

Σ^n = set of all string of length n.

Cardinality: The number of elements in a set

→ Cardinality of $\Sigma^0 = 2^0 = 1$

→ Cardinality of $\Sigma^1 = 2^1 = 2$

∴ Cardinality of $\Sigma^n = 2^n$

$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \ldots$

$= \{\epsilon\} \cup \{a, b\} \cup \{ab, ba, aa, bb\} \cup \ldots$

= set of all possible strings of all lengths over $\{a, b\}$

= It is infinite set.

→ Σ^* is called kleene closure

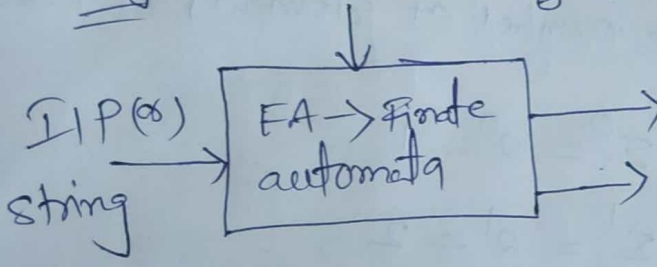
$\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \Sigma^4 \ldots$

→ It can generate all possible strings of all lengths except "0" length.

$$\Sigma^* = \Sigma^+ \cup \{\epsilon\}$$

→ Using TOC we need to develop a machine that accepts the string and check whether the ~~string~~ given string belongs to that language (or) not

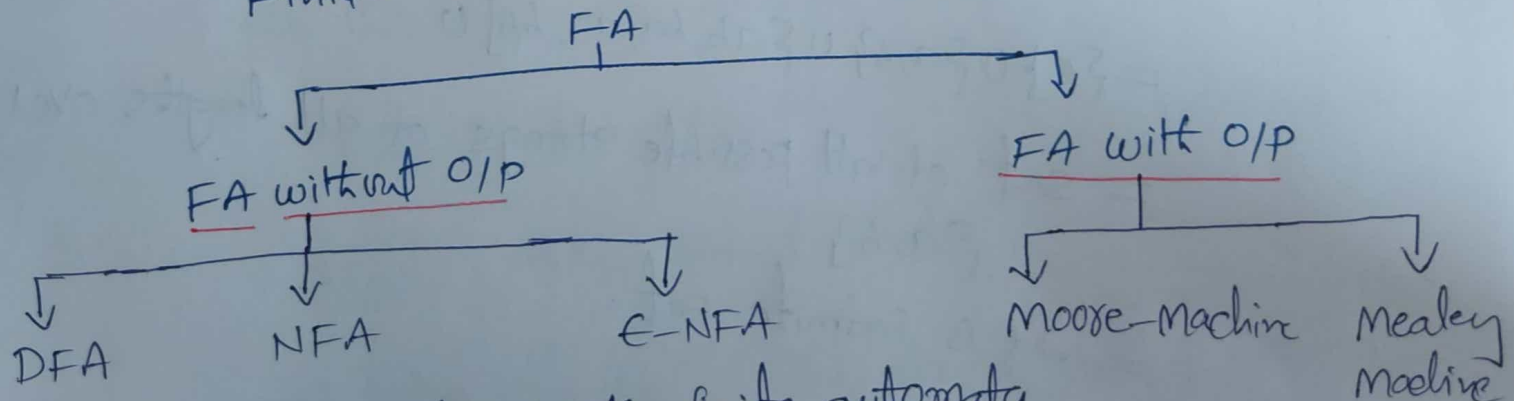
Eg: L_1 = set of all strings starting & ending with b over $\{a, b\}$



→ If the i/p is aaa, then it is rejected

→ If the i/p is baab then it is accepted

→ Finite Automata is divided into two categories



DFA – Deterministic finite automata

NFA – Non-Determini finite automata

ϵ-NFA – ϵ- " " "

① DFA:- It has only one path for given specific i/p from current state to next state.

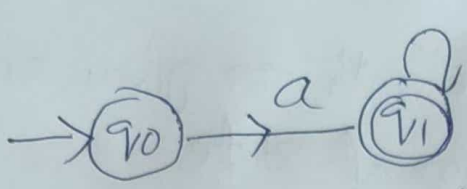
→ It has 5 tuples.
→ It has limited memory.
→ we can do calculation easily using DFA
→ ~~the~~ 5 tuples of DFA are
$(Q, \Sigma, q_0, \delta, F)$

$Q \rightarrow$ No of ~~states~~ in DFA
$\Sigma \rightarrow$ Inputs for the FA
$q_0 \rightarrow$ Initial state
$F \rightarrow$ Final state
$\delta \rightarrow Q \times \Sigma = Q$ → V. Imp
$F \subseteq Q$

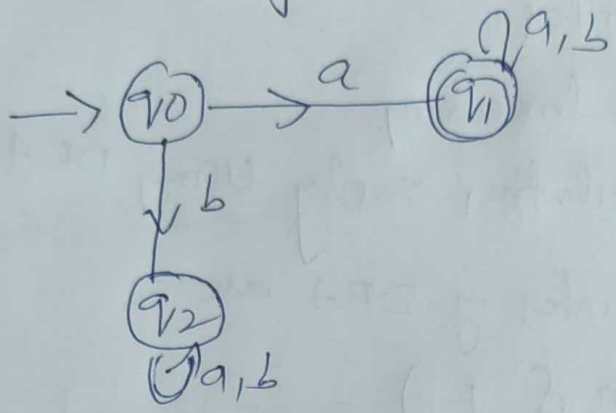
Eg1:- L_1 = set of all strings starting with "a" on over $\{a, b\}$
For this language design DFA.

Sol:- $= \{a, ab, aa, aaa, aba, \ldots\}$ → Infinite set

→ (q0) take Initial state
→ (q0) —a→ (q1) (This for "a" only) Step-1
→ (q0) —a→ (q1) —b↺ (This for ab)



If string start with "b"



Here 5-Tuples are

$Q := \{q_0, q_1, q_2\}$

$\Sigma = \{a, b\}$

$q_0 = \{q_0\}$

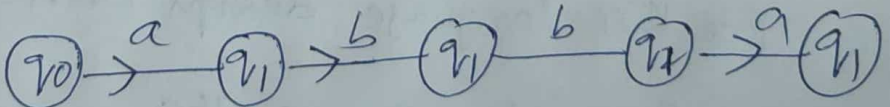
$F = \{q_1\}$

Transisation Diagram

$\delta: Q \times \Sigma = Q$

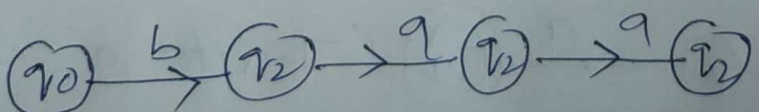
Q \ Σ	a	b
→q0	q1	q2
*q1	q1	q1
q2	q2	q2

Eg1: If string abba, is it accepted (or) rejected

Initial state 

It is accepted

If the string is baa

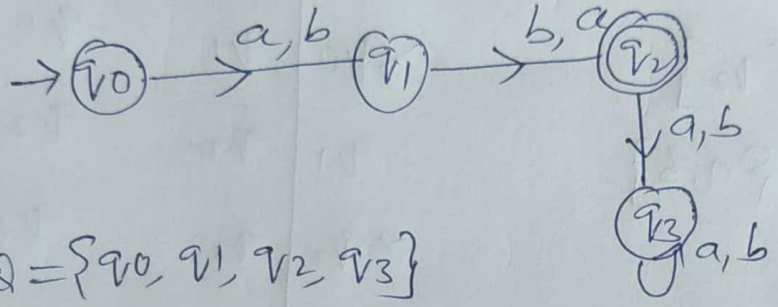


Finally we reached to "q_2" but q_2 is not final state so It is rejected.

Eg2:- Construct DFA for the set of all strings of length 2 on over $\{a,b\}$

Sol:- L_1 = set of all strings of length 2 over $\Sigma=\{a,b\}$

$=\{ab, aa, bb, ba\}$



$Q=\{q_0, q_1, q_2, q_3\}$

$\Sigma=\{a,b\}$

$q_0=\{q_0\}$

$F=\{q_2\}$

Transition Diagram

Q \ Σ	a	b
→ q_0	q_1	q_1
q_1	q_2	q_2
q_2	q_3	q_3
q_3	q_3	q_3

→ If the string is "aba"

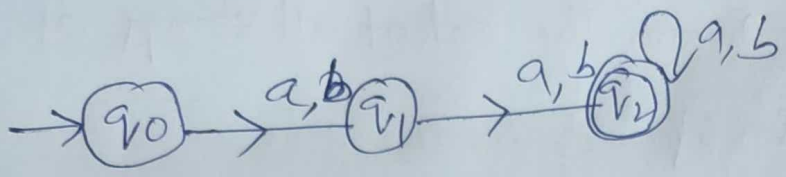
∴ It is rejected b/c It has length "3"

Eg3:- Construct DFA for the set of all strings of atleast length 2. $\Sigma=\{a,b\}$

Ans:- $=\{aa, ba, bb, ab, aaa, baaa \ldots\ldots\}$

$=\{a, b, \epsilon\}$ not allowed.

→ First take minimal string i.e aa



$Q = \{q_0, q_1, q_2\}$

$q_0 = \{q_0\}$

$F = \{q_2\}$

$\Sigma = \{a, b\}$

Transition Diagram

Q \ Σ	a	b
→q_0	q_1	q_1
q_1	q_2	q_2
*q_2	q_2	q_2

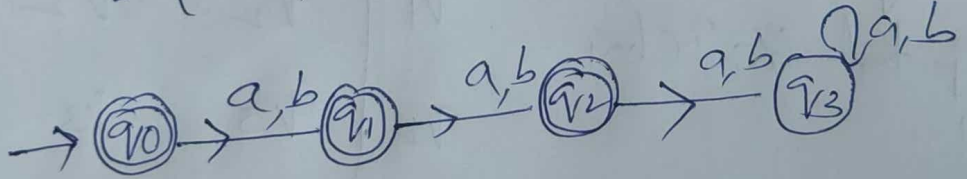
(3) L_3 = Set of all strings of atleast length 2 over $\{a, b\}$

Design DFA

Ans:

$= \{\epsilon, a, b, aa, ab, ba, bb\}$ accepted

$= \{aaa, bba, bab \ldots\}$ Rejected.



$Q = \{q_0, q_1, q_2, q_3\}$

$q_0 = \{q_0\}$

$F = \{q_0, q_1, q_2\}$

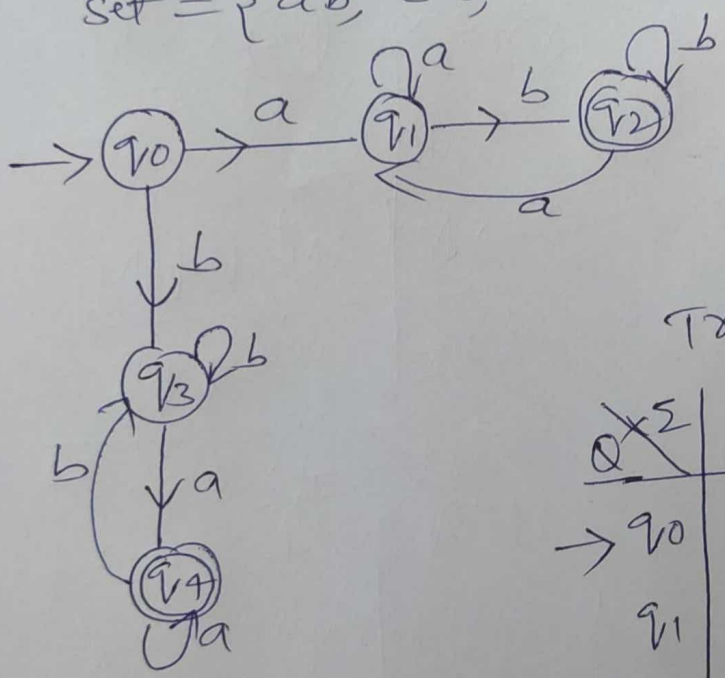
$\Sigma = \{a, b\}$

Note:

* No of states for length n is $n+2$
* No of states for length atleast n is $n+1$
* No of states for length atmost n is $n+2$

Eg4: Design DFA for set all strings starting and ending with different symbol over $\{a,b\}$.

Ans: set = {ab, ba, abab, aab, baaa ....}



Transition Diagram

Q \ Σ	a	b
→ q_0	q_1	q_3
q_1	q_1	q_2
*q_2	q_1	q_2
q_3	q_4	q_3
*q_4	q_4	q_3

$Q = \{q_0, q_1, q_2, q_3, q_4\}$

$\Sigma = \{a, b\}$

$F = \{q_2, q_4\}$

$q_0 = \{q_0\}$

$\delta = Q \times \Sigma = Q$

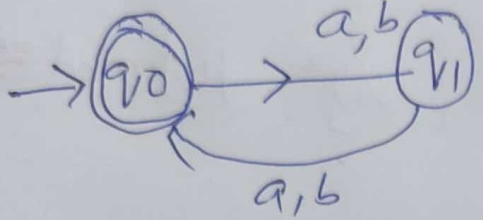
Eg5) Construct DFA for $w \in \{a,b\}$ ~~$|w| \bmod 2 = 0$~~ $|w| \bmod 2 = 0$

Sol:-

Set. $\Sigma = \{a,b\}$

$L = \{\epsilon, aa, bb, ab, babb, aabab \ldots\}$

↓ Rejected



Transition diagram

	a	b
→ q0	q1	q1
*q1	q0	q0

if $w = aaba$

→ Check this string is accepted (or) rejected

$\delta(q_0, aaba)$

$\delta(q_1, aba)$

$\delta(q_0, ba)$

$\delta(q_1, a)$

$\delta(q_0, \epsilon)$

↓ ↘ string is ended

If q_0 is final state, It is accepted

if $w = aba$

$\delta(q_0, aba)$

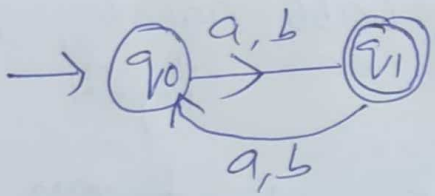
$\delta(q_1, ba)$

$\delta(q_0, a)$

$\delta(q_1, \epsilon)$ ⇒ ~~final q~~ Now q_1 is non final state so this string is rejected.

Eg6: Construct minimal DFA, $W \in \{a,b\}$ $|W| \bmod 2 = 1$ (2)

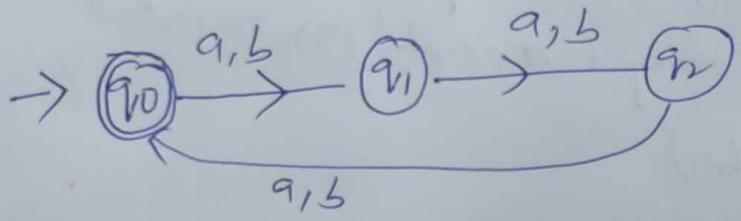
Sol: set = {a, b, aaa, bab, ababa ....}



If $w = abab \rightarrow$ It is rejected

Eg7: Construct minimal DFA for $W \in \{a,b\}^*$ $|W| \bmod 3 = 0$

Sol: set = {∈, aaa, bba, aab, aababaa ....}
bbb

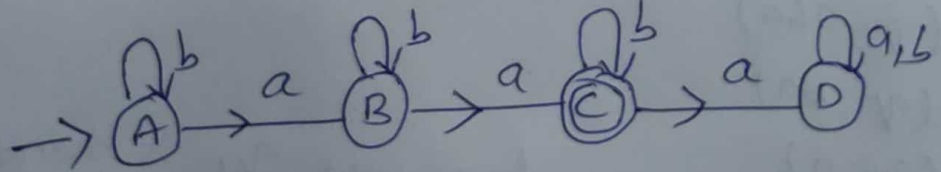


Note:

$\|W\| \bmod n = 0$	then	no. of states $= n$
$\|W\| \bmod n = 1$	"	" " $= n$

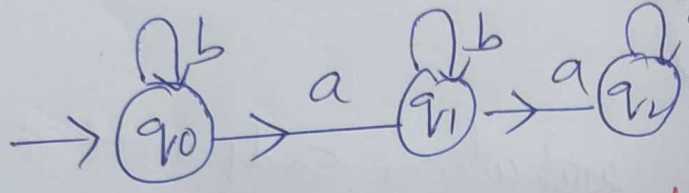
Eg8: Construct minimal DFA $W \in \{a,b\}^*$ no of "a" are 2 only (∞) $n_a(W) = 2$

Sol: L = {aa, baa, aba, aab ....}



Eg9:- Construct minimal DFA for $w \in \{a,b\}^*$ number of 'a' are atleast 2 i.e $n_a(w) \geq 2$

Sol:- $L = \{aa, aab, baab, aaab, baaab, \ldots\}$

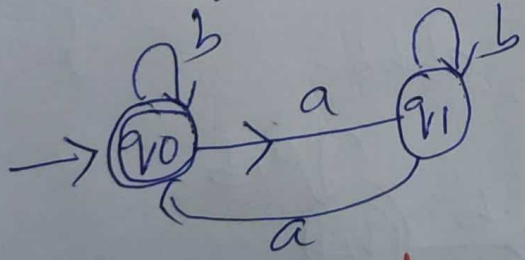


Eg10:- Construct minimal DFA for $w \in \{a,b\}^*$ number of 'a' are atmost 2 i.e $n_a(w) \leq 2$

Sol:- Home work.

Eg11:- Construct minimal DFA for $w \in \{a,b\}^*$ no. of a are divisible by 2 $n_a(w) \bmod 2 = 0$

Sol:- set = $\{\in, aa, b, bba, aab, bbbaa \ldots\}$

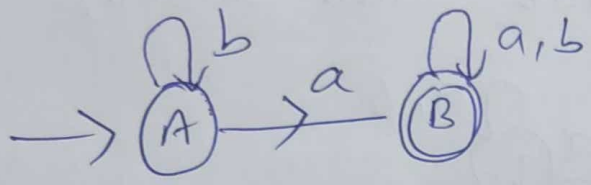


Eg12:- Construct minimal DFA which accepts set of all string over $\{a,b\}$ where each string start with an 'a'

Eg 13: Construct DFA for set of all string contains "a"

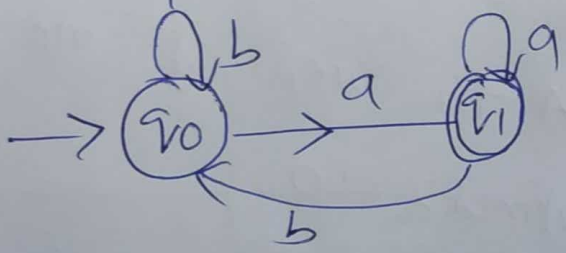
$\Sigma = \{a, b\}$

Sol: $L = \{a, aa, ab, ba, bba \ldots\}$



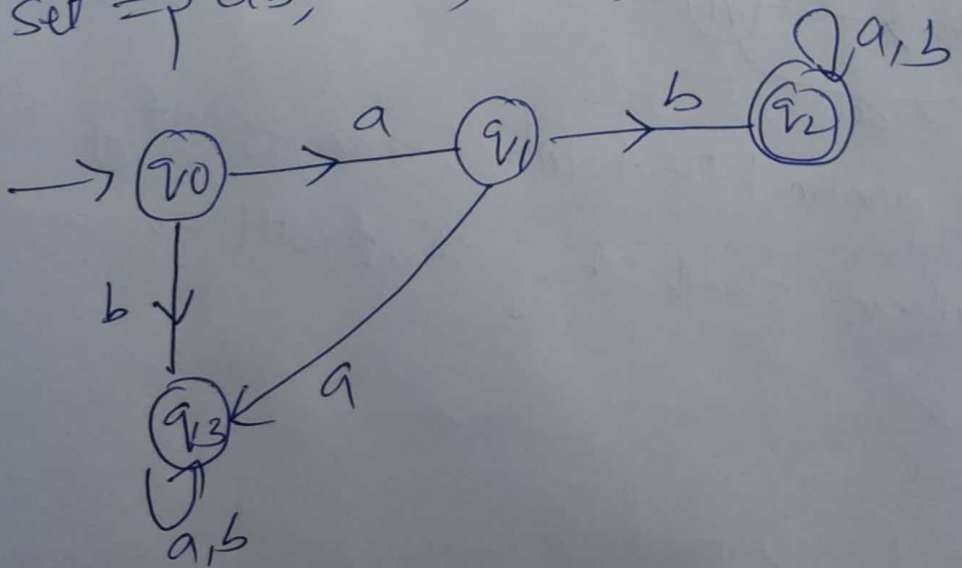
Eg 14: DFA for strings which ends with "a" on over $\{a, b\}$

Sol: set $= \{a, ba, aaa, bbba \ldots\}$



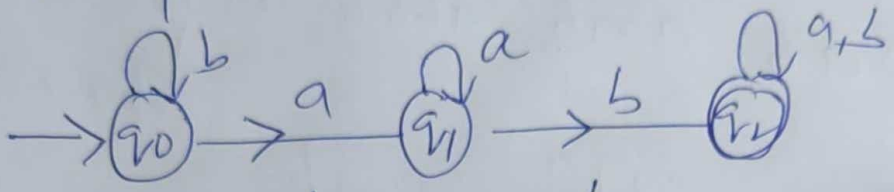
E15: Construct a DFA which accepts set of all string over $\{a, b\}$ which start with "ab" }

Sol: set $= \{ab, abb, aba, abbb \ldots\}$



Eg6: Construct a DFA which accepts set of all strings over {a, b} which contains "ab"

Sol: $L = \{ab, abba, bbab, bab \ldots\}$



W = baab

Transition Diagram

	a	b
→ q0	q1	q0
q1	q1	q2
q2	q2	q2

$\delta(q_0, baab)$

$\delta(q_0, aab)$

$\delta(q_1, ab)$

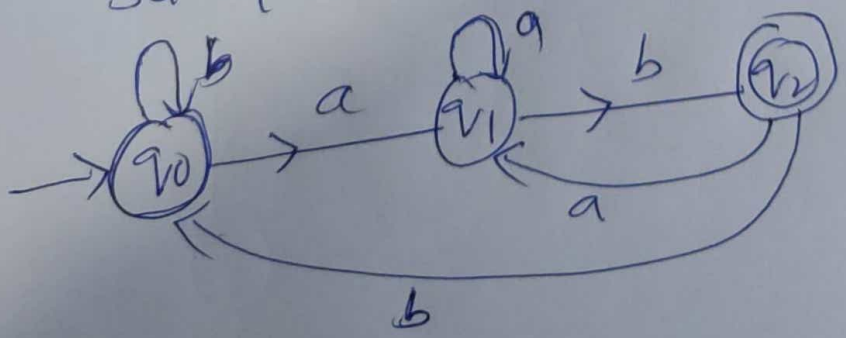
$\delta(q_1, b)$

$\delta(q_2, \epsilon)$

↳ It is final state so It is accepted

Eg 17: Construct a DFA ~~with~~ for end with "ab" on over {a, b}

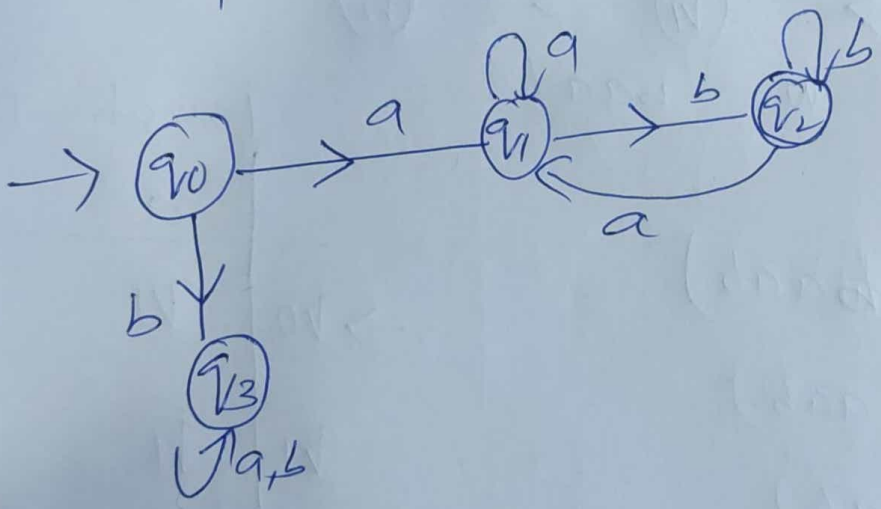
Sol: set = {ab, bab, bbab, aab, ...}



abbb

18Q:- Construct a DFA which accepts set of all strings over {a,b} which starts with "a" and end with "b"

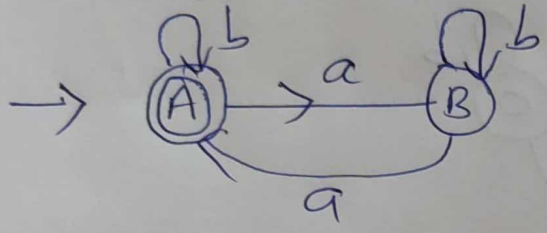
Sol:- set = {ab, aaab, abbb, aaaba - - -}
aaaba → not accepted



(19) Construct a minimal DFA which accepts set of all strings over {a,b} in which number of a's are division by 2 & number of b's are divisible by 2.

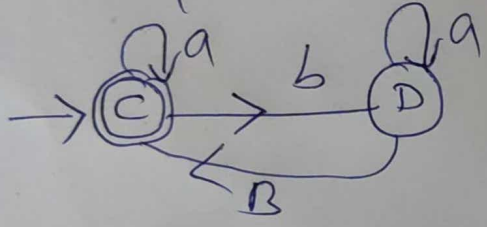
Sol: First Draw DFA for "a"

set = {∈, aa, baa, aba . . . }



next Draw DFA for "b"

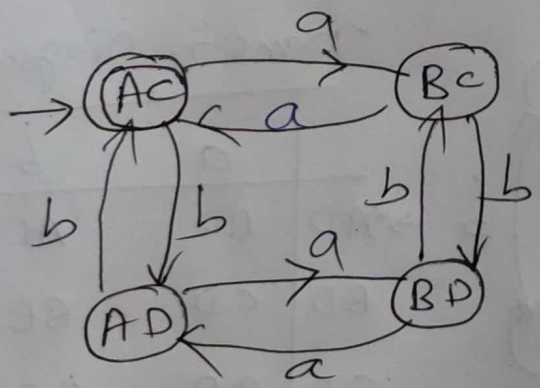
set = {∈, bb, bbbb, abbbba . . .

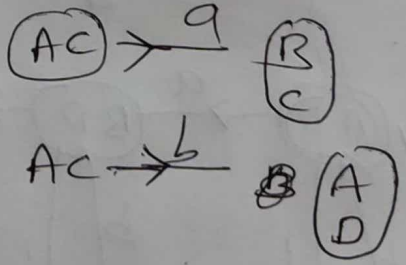


→ Combinid both "a" & "b"

{A,B} X {C,D} = {AC, AD, BC, BD}

Take AC, AD, BC, BD are states.



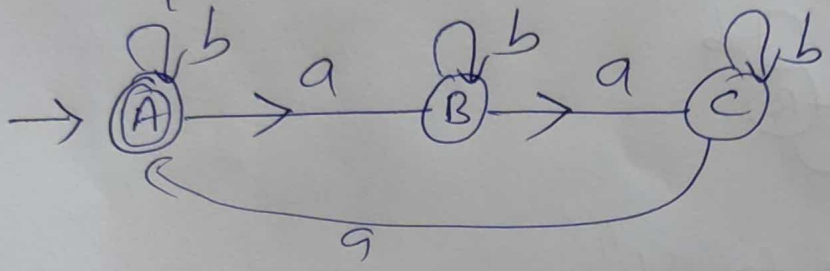


set = {aa, abba, . . . abbaa . . . }

(20) construct a minimal DFA which accepts set of all strings over {a,b} in which number of a's are divisible by 3 and number of b's are divisible by 2

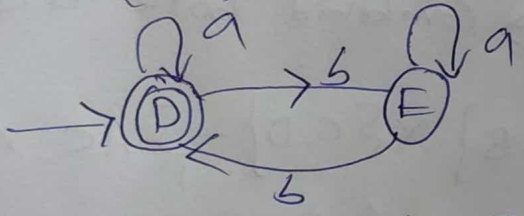
Sol:- First draw number of 'a' are divisible by 3

$= \{ \epsilon, aaa, aaab, aaabaaa \cdots \}$

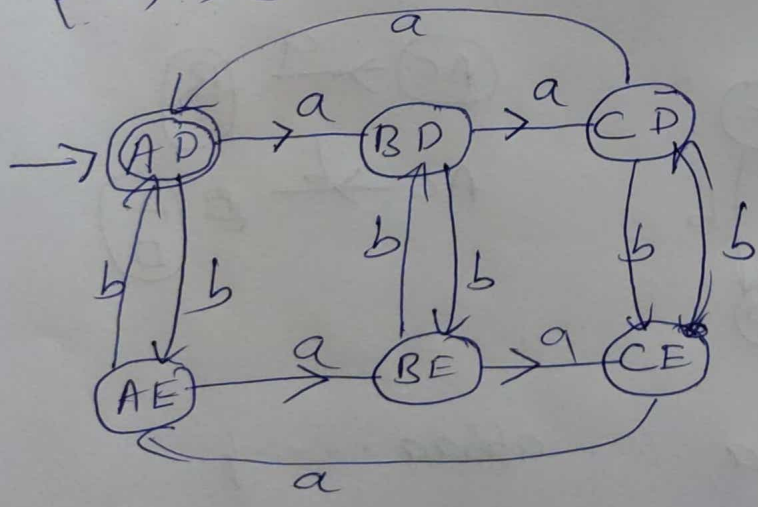


next draw number of "b" are Divisable by 2

$= \{ \epsilon, bb, abb, bab \cdots \}$



$\{A, B, C\} \times \{D, E\} = \{AD, BD, CD, AE, BE, CE\}$



Transition Diagram

	a	b
→ *AD	BD	AE
BD	CD	BE
CD	AD	CE
AE	BE	AD
BE	CE	BD
CE	AE	CD

(21) Construct a DFA which accepts set of all strings over $\{0,1\}$, which when interpreted as binary number is divisible by 2

Sol. $\Sigma = \{0,1\}$ $w \in \{0,1\}^*$

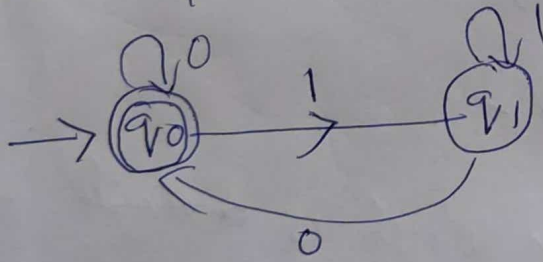
Introduction about binary numbers

101101

$= 5\times2+1 = 11\times2+0 = 22\times2+1 = 45$

→ If you divide any number by "2" we get '0' (or) '1' Remender, so we have 2 states for above problem

$= set = \{0, 10, 010, 1010 \ldots\}$

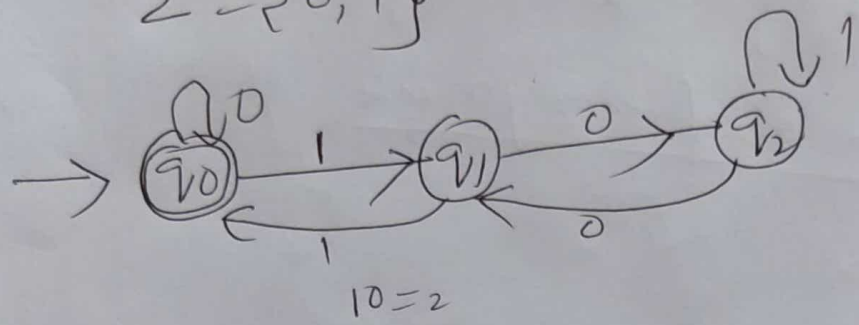


Transition Diagram

	0	1
→ q0*	q0	q1
q1	q0	q1

(22) Construct minimal DFA which accepts set of all strings over $\{0,1\}$ which when interpreted as binary number is divisible by "3"

Sol:- $\Sigma = \{0, 1\}$



10 = 2

Transition Diagram

	0	1
→ *q0	q0	q1
q1	q2	q0
q2	q1	q2

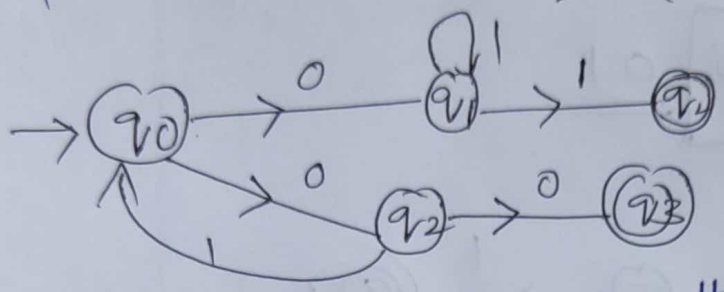
(23) DFA of Binary number which is divisible by 4

NFA (Non-Deterministic finite automata)

→ The finite automata ale called NFA when there exist many paths for specific input from the current state to the next state

$\Sigma = \{0, 1\}$

Eg:



→ It is easy to construct an NFA than DFA for a given language.

→ NFA also have five states same as DFA but with different transition function that is

$$\delta : Q \times \Sigma \to 2^Q$$

$M = (Q, \Sigma, q_0, F, \delta)$

Q: finite set of states

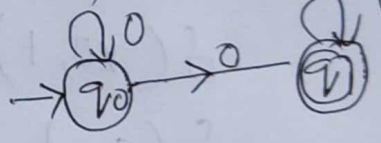
Σ: finite set of I/P symbol

q_0: Initial state

F: Final state

δ: Transition function

Eg:



$\Sigma = \{0, 1\}$

In NFA q_0 on "0" we have 2^Q options that are total

$= 2^2 = 4$

$q_0 \xrightarrow{0} q_0$

$q_0 \xrightarrow{0} q_1$

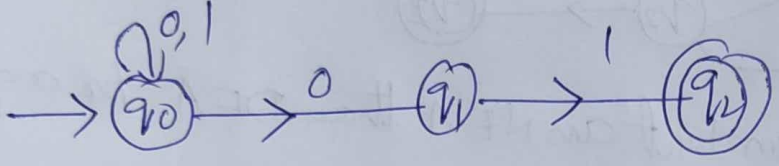
$q_0 \xrightarrow{0} \phi$

$q_0 \xrightarrow{} q_0 q_1$

Eg1: Construct NFA with $\Sigma = \{0, 1\}$ accepts all strings ending with 01

Sol: Set = {01, 101, 0001, 001, . . . }

Anything either '0' or '1' 01



Eg: If string w = 001

$\delta(q_0, 001) \Rightarrow$

option 1: $\delta(q_0, 01)$

$\delta(q_0, 1)$

$\delta(q_0)$

$\rightarrow q_0$ – non final state

X

option 2: $\delta(q_0, 001)$

$\delta(q_0, 01)$

$\delta(q_1, 1)$

$\delta(q_2, \epsilon)$

↓

So, Here q_2 is final state so the string is accepted

option 3: $\delta(q_0, 001)$

$\Rightarrow \delta(q_1, 01)$

→ Here $(q_1, 0)$ not there so stop the transition and check the total string is end or not. If not end say not accept X

Note: Any of the option is accepted the string is belongs to that Language

$Q = \{q_0, q_1, q_2\}$, $\Sigma = \{0, 1\}$ $q_0 = \{q_0\}$ $F = \{q_2\}$

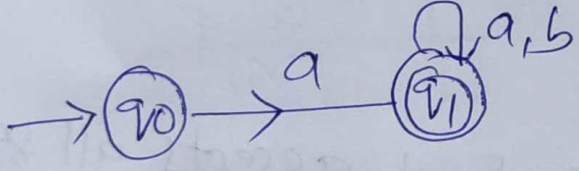
$\delta : Q \times \Sigma = 2^Q$

	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	q_0
q_1	ϕ	q_2
q_2	ϕ	ϕ

Eg2:- Design an NFA in which all string ~~Containg~~ staring with a $\Sigma = \{a, b\}$

Sol:- $= \{$~~ab, a, a~~

$= \{a, ab, aa, abb \ldots -$

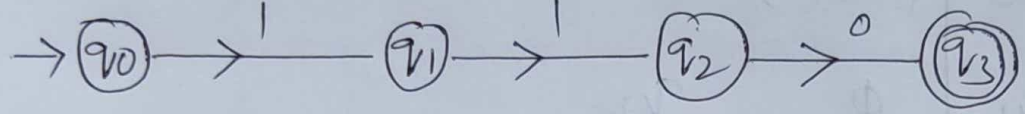


Transition Diagram $\delta : Q \times \Sigma = 2^Q$

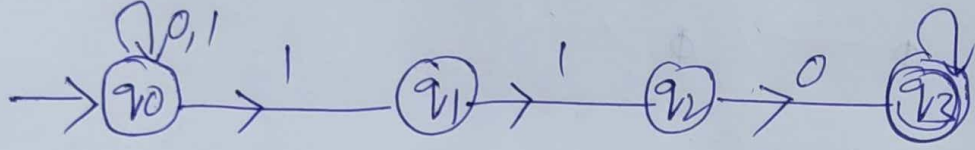
	a	b
$\rightarrow q_0$	q_1	ϕ
q_1	q_1	q_1

Eg3:- Design an NFA in which all string Contain a Substring 110

Sol:- = {110, 0110, 1101, 00110, 11101 . . . }



Step-1

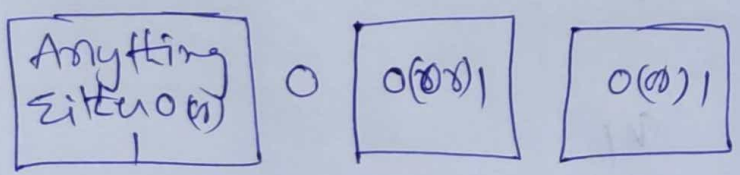


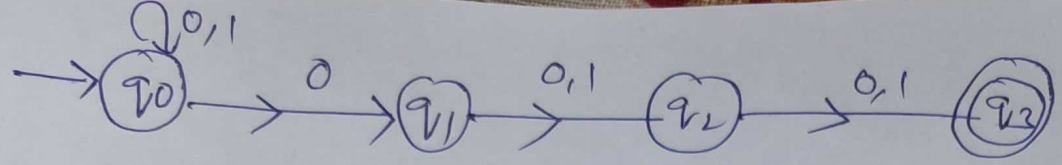
Transition Diagram

	0	1
→ q0	q0	{q0, q1}
q1	ϕ	q2
q2	q3	q3
q3	q3	q3

Eg4:- Design an NFA with Σ = {0, 1} accepts all string in which the third symbol from the right end is always "0"

Sol:-





If $w = 1011$

$\delta(q_0, 1011)$

$\delta(q_0, 011)$

$\delta(q_0, 11)$ → $\delta(q_0, 1)$ → $\delta(q_0, \epsilon)$ → ✗

$\delta(q_1, 11)$ → $\delta(q_2, 1)$ → $\delta(q_3, \epsilon)$

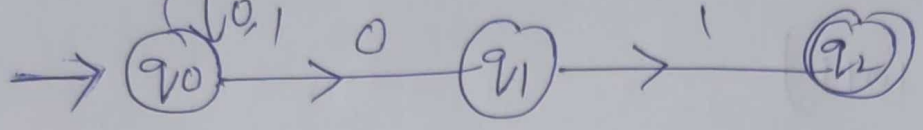
q_3 – final state so given string is accepted

Transition Diagram

	0	1
→ q_0	$\{q_0, q_1\}$	q_1
q_1	q_2	q_2
q_2	q_3	q_3
~~q_3~~	ϕ	ϕ

(6) ~~Conv~~ Convert NFA to DFA

Eg1:- Convert NFA to DFA for following FA



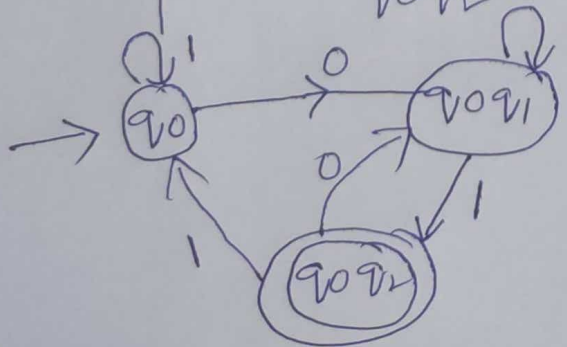
Sol:- Step1:- Draw the Transition Diagram first for given NFA

	0	1
→ q_0	$\{q_0, q_1\}$	q_0
q_1	ϕ	q_2
*q_2	ϕ	ϕ

Convertion of NFA to DFA, start with "q_0"

	0	1
→ $[q_0]$	$[q_0 q_1]$	$[q_0]$
$[q_0 q_1]$	$[q_0 q_1]$	$[q_0 q_2]$
*$[q_0 q_2]$	$[q_0 q_1]$	$[q_0]$

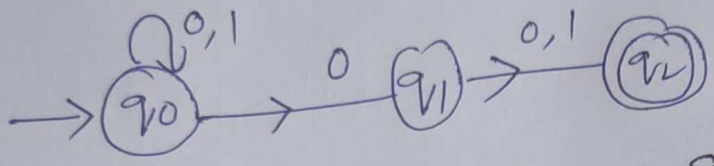
Step3:- Newly formed states are q_0, q_0q_1, q_0q_2



⇒ It is DFA, is equivalent to NFA

Eg2:- Convert NFA to DFA "all strings in which the second symbol from the right end is always "0"

Sol:- (0+1)* 0 (0+1)



Step1:- Transition Diagram

	0	1
→q0	{q0, q1}	q0
q1	q2	q2
q2	ϕ	ϕ

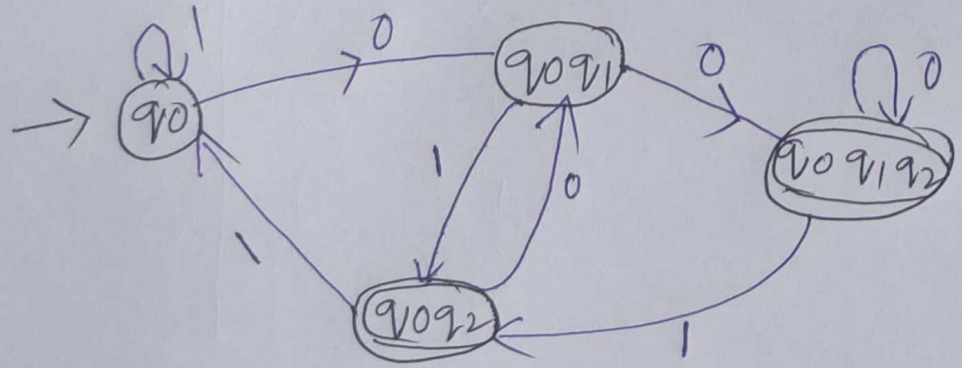
Step2:- Conversion

	0	1
→q0	[q0q1]	[q0]
[q0q1]	[q0q1q2]	[q0q2]
*[q0q1q2]	[q0q1q2]	[q0q2]
*[q0q2]	[q0q1]	[q0]

→ After converting NFA to DFA the newly states are

q0, (q0q1), (q0,q1q2), (q0q2)

Draw the DFA



∈-NFA

→ ∈-NFA is same as NFA except there would be an edge labeled "∈" b/w two states which allows the transition from one state to another state without receiving an input symbol.

→ ∈-NFA has 5 tuples that are

$= \{Q, \Sigma, \delta, q_0, F\}$

only δ is changing i.e

$$\delta : Q \times [\Sigma \cup \{\epsilon\}] = 2^Q$$

$F \subseteq Q$

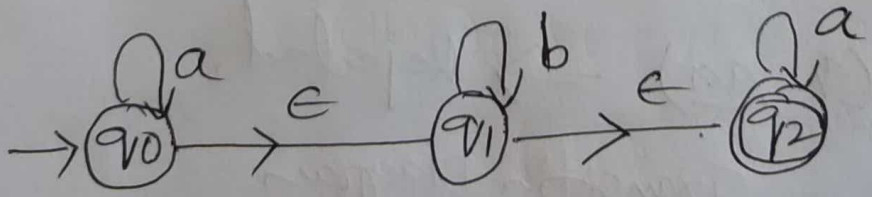
$q_0 \subset Q$

Advantages:

(1) We can use ∈-NFA in regular expression

(2) Without giving any i/p, we can go from one state to another state using ∈-NFA

Ex:



$\delta(q_0, a) = q_0$, $\delta(q_0, \epsilon) = q_1$, $\delta(q_1, b) = q_1$

$w = abba$

$\delta(q_0, abba) \Rightarrow \delta(q_0, \epsilon bba) \Rightarrow \delta(q_1, bba)$

$\delta(q_1, ba) \Rightarrow \delta(q_1, a) \Rightarrow \delta(q_2, a) \Rightarrow \delta(q_2, \epsilon)$

Fin

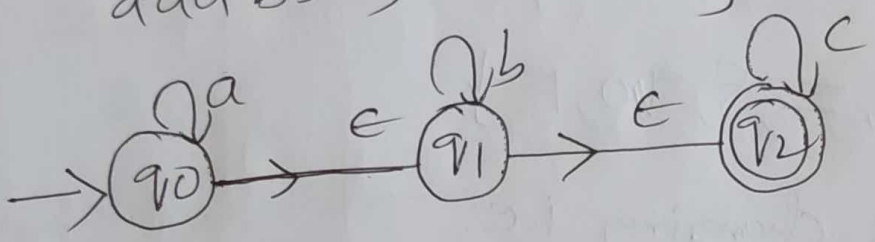
Eg1:- Design ϵ-NFA for a string $L = \{a^m b^n c^k / m, n, k \geq 0\}$ on over $\{a, b, c\}$

Sol:- $L = \{a^m b^n c^k / m, n, k >= 0\}$

$= \{\epsilon, a, b, c, ab, ac, ba, cab \rightarrow$ not allow.

(ba ↓ not allowed)

$aaabbc, \ldots\}$



If $w = abc$

$\delta(q0, abc) \Rightarrow \delta(q0, bc) \Rightarrow \delta(q1, bc)$

$\delta(q1, c) \Rightarrow \delta(q2, c) \Rightarrow \delta(q2, \epsilon)$

↓ string is compld. & $q2$ is final state

If $w = baa$

$\delta(q0, baa) = \delta(q1, baa) = \delta(q1, aa)$

$\delta(q2, aa) \Rightarrow$ Rejected.

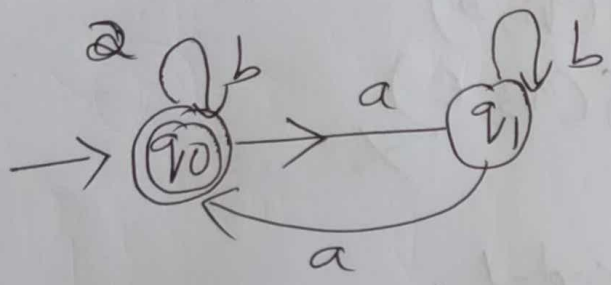
Transition Diagram

	a	b	c	ε
→q0	q0	ϕ	ϕ	q1
q1	ϕ	q1	ϕ	q2
*q2	ϕ	ϕ	q2	ϕ

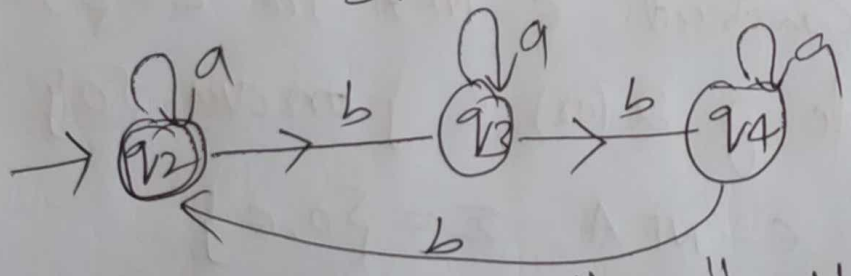
Eg2: set of all strings in which no. of a's are divisible by 2 (or) b's are divisible by 3 over $\{a, b\}$.

Sol: set = {b, a, aab, aba, bbba ...}

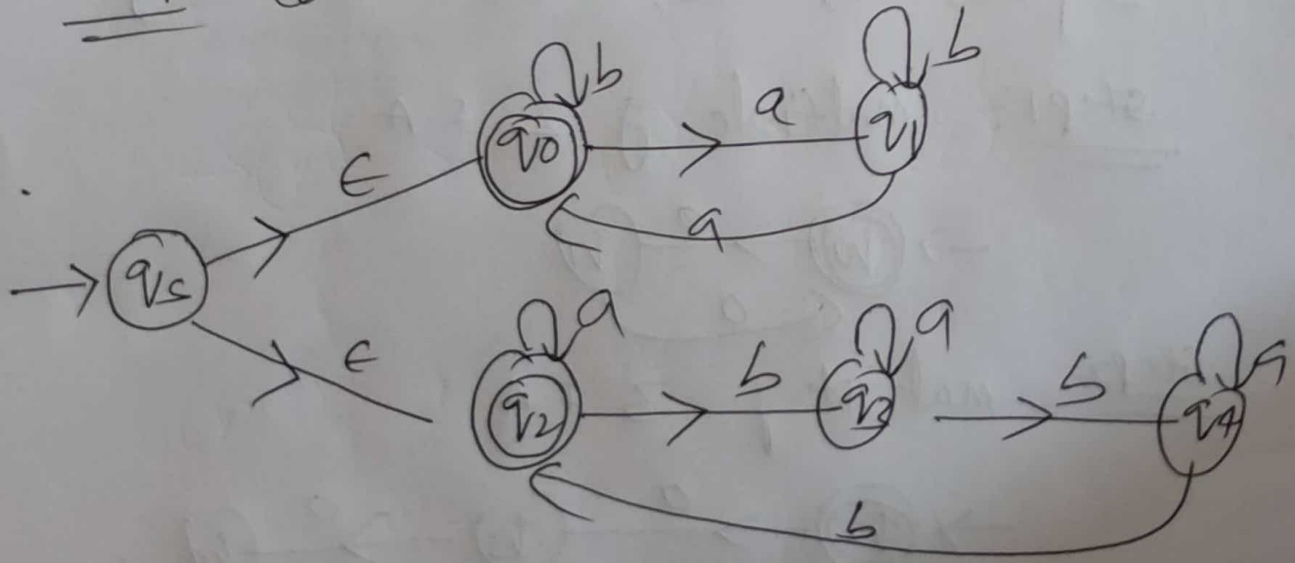
Step1: First construct no of "a" are divisible by 2



Step2: Construct no. of "b" are divisible by 3.



Step3: Combinde both "FA" with "ϵ"



If $w = aaab$

$\delta(q_0, aaab)$

$\delta(q_0, aab)$

$\delta(q_1, ab)$

$\delta(q_0, b)$

$\delta(q_0, \epsilon)$

↓

Final state

So given string is accepted

$\delta(q_2, aab)$

↓

$\delta(q_2, ab)$

$\delta(q_2, b)$

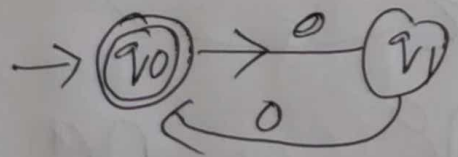
$\delta(q_3, \epsilon)$ ✗

Eg3: Construct ϵ-NFA for $L = \{0^k \mid k$ is multiple of 2 (or) 3 $\}$ on $\Sigma = \{0\}$

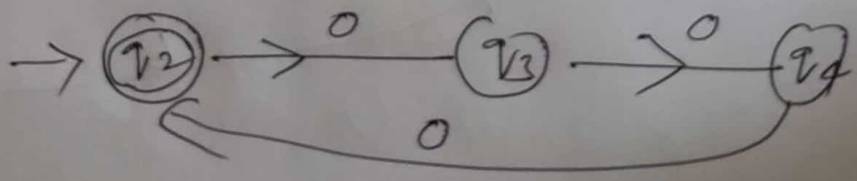
Sol: In ϵ-NFA $\Sigma = \{0, \epsilon\}$

$= \{00, 000, 0000, 00000 \ldots\}$

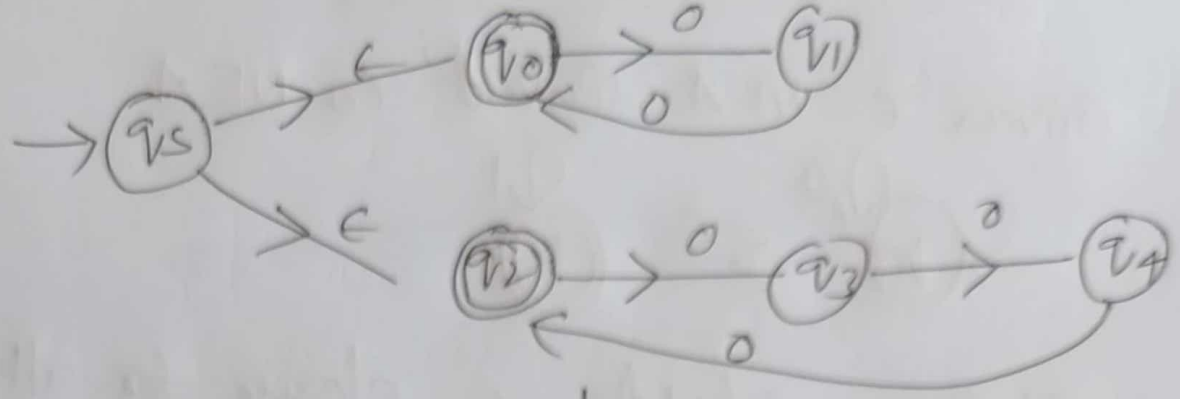
Step1: multiple of 2 FA



Step2: multiple of 3 FA



→ Combin both FA with ∈



If w = 0000

δ(qs, 0000)

δ(q0, 0000) δ(q2, 0000)

δ(q1, 000)
δ(q0, 00)
δ(q1, 0)
δ(q0, ϵ)
accept

If w = 00000

δ(qs, 00000)

δ(q0, 00000) δ(q2, 00000)

δ(q1, 0000)
δ(q0, 000)
δ(q1, 00)
δ(q0, 0)
δ(q1, ϵ)
Not accept

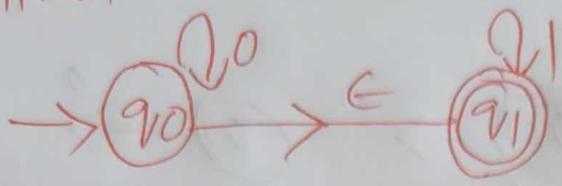
δ(q3, 0000)
δ(q4, 000)
δ(q2, 00)
δ(q3, 0)
δ(q4, ϵ)
Not accept.

Transition Diagram

	0	ϵ
→qs	ϕ	{q0, q2}
*q0	q1	ϕ
q1	q0	ϕ
*q2	q3	ϕ
q3	q4	ϕ
q4	q2	ϕ

Convert ∈ NFA to NFA

Eg1: Given ∈-NFA Convert to NFA



Sol:

Step 1: Calculate ∈-closure for all states.

So $\in$-closure$(q_0) = \{q_0, q_1\}$

$\in$-closure$(q_1) = \{q_1\}$

Step 2: Find the Transition function.

$$\bar{\delta}(q_0, w) = \in\text{-closure}(\delta(\bar{\delta}(q_0, \in), w))$$

$$\bar{\delta}(q, \in) = \in\text{-closure}(q)$$

$\therefore$

$$\boxed{\bar{\delta}(q_0, w) = \in\text{-closure}(\delta(\in\text{-closure}(q), w))}$$

$$\bar{\delta}(q_0, 0) = \in\text{-closure}(\delta(\in\text{-closure}(q_0), 0)$$

$$= \in\text{-closure}(\delta(\{q_0, q_1\}, 0)$$

$$= \in\text{-closure}(\delta(q_0, 0) \cup \delta(q_1, 0))$$

$$= \in\text{-closure}(q_0 \cup \phi)$$

$$= \in\text{-closure}(q_0) \cup \in\text{-closure}(\phi)$$

$$= \{q_0, q_1\} \cup \{\phi\} = \{q_0, q_1\}$$

$$\bar{\delta}(q_0,1) = \epsilon\text{-closure}[\delta(\epsilon\text{-closure}(q_0),1)]$$

$$= \epsilon\text{-closure}[\delta(q_0,q_1),1)]$$

$$= \epsilon\text{-closure}[\delta(q_0,1) \cup \delta(q_1,1)]$$

$$= \epsilon\text{-closure}[\phi \cup q_1]$$

$$= \epsilon\text{-closure}(\phi) \cup \epsilon\text{-closure}(q_1)$$

$$\therefore \bar{\delta}(q_0,1) = \phi \cup q_1 = \{q_1\}$$

$$\bar{\delta}(q_1,0) = \epsilon\text{-closure}(\delta(\epsilon\text{-closure}(q_1),0))$$

$$= \epsilon\text{-closure}(\delta(q_1,0))$$

$$\bar{\delta}(q_1,0) = \epsilon\text{-closure}(\phi) = \phi$$

$$\bar{\delta}(q_1,1) = \epsilon\text{-closure}(\delta(\epsilon\text{-closure}(q_1),1))$$

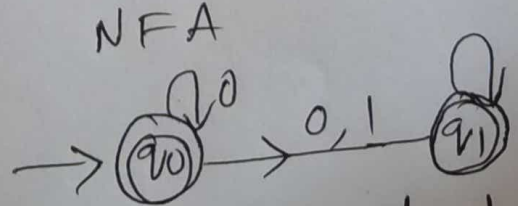
$$= \epsilon\text{-closure}(\delta(q_1,1))$$

$$= \epsilon\text{-closure}(q_1) = q_1$$

Transition Table for NFA

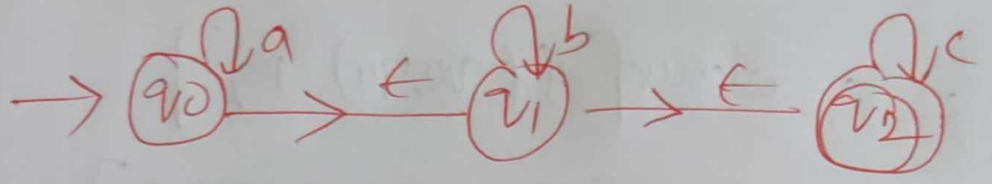
	0	1
→ q_0	$\{q_0,q_1\}$	$\{q_1\}$
* q_1	ϕ	$\{q_1\}$

NFA



Final state: Each state closure that contains final state then that state also become final state.

Eg2:- Given ∈-NFA Convert to NFA



* Minimization of DFA

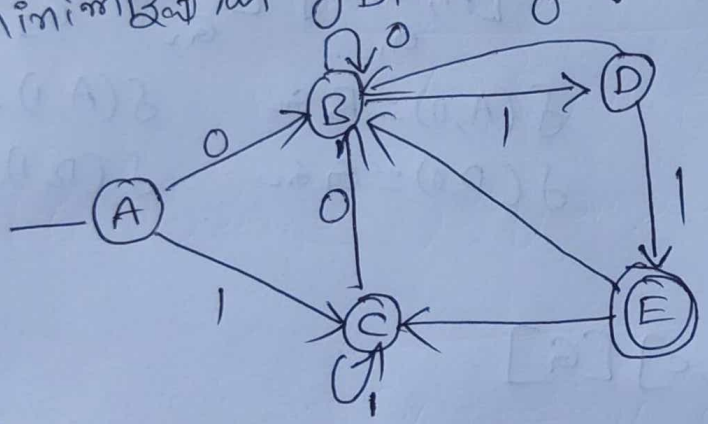
→ Minimization of DFA is required to obtain the minimal version of any DFA which consists of the minimum number of states possible.

· Equivalent

Two states "A" and "B" are said to be equivalent if

$\delta(A,X) \to F$ and $\delta(B,X) \to F$ (or) $\delta(A,X) \to F$ and $\delta(B,X) \to F$ where X is any i/p string.

Eg1: Minimization of DFA for given diagram.



Step1:

	0	1
→ A	B	C
B	B	D
C	B	C
D	B	E
*E	B	C

Step2:-

0 equivalent: $\{A,B,C,D\}$ $\{E\}$ → Separate final state & non final state
G_1 G_2

1 equivalent: $[A,B,C]$ $[D]$ $[E]$
G_1 G_2 G_3

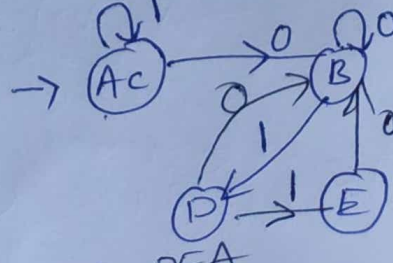
$\delta(A,0)=B$ and $\delta(A,1)=C$ Both are going same group so B stay with A only
$\delta(B,0)=B$ $\delta(B,1)=D$

2-equivalent: $[A,C]$ $[B]$ $[D]$ $[E]$

$\delta(A,1)=C-G_1$

$\delta(B,1)=D-G_2$

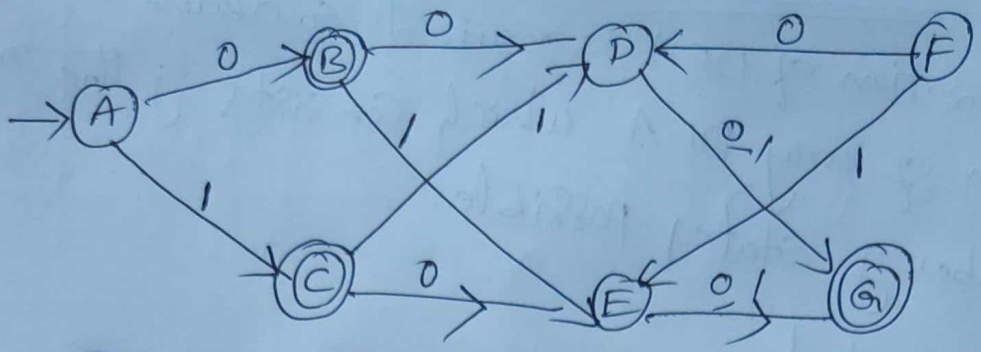
3-equivalent: $[A,C]$ $[B]$ $[D]$ $[E]$



→ DFA

Minimization of DFA

Eg2:-



Sol:-

Transition Diagram

	0	1
A	B	C
*B	D	E
*C	E	D
D	G	G
E	G	G
*G	G	G

(1) 0-equivalent

$[A, D, E]_{G_1}$ $[B\ C\ G]_{G_2}$

1st-equivalent —

$[A, D, E]_{G_1}$ $[B, C]_{G_2}$ $[G]_{G_3}$

$\delta(A,0) = B\ G_2$ $\delta(A,1) = C$

$\delta(D,0) = G\ G_2$ $\delta(D,1) = G$

2nd-equivalent —

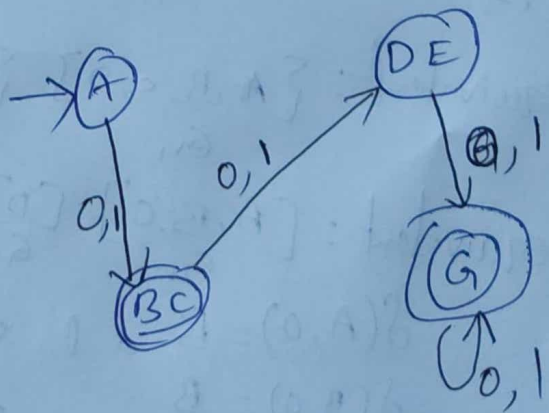
$[A]$ $[DE]$ $[B, C]$ $[G]$

$\delta(A,0) = B - G_2$

$\delta(D,0) = G - G_3$

3rd-equivalent

$[A]$ $[DE]$ $[B, C]$ $[G]$



PO	1	2	3	4	5	6	7	8	9	10	11	12
Level		H	M		M							

Syllabus for B. Tech. III Year II semester
Computer Science and Engineering
AUTOMATA THEORY AND COMPILER DESIGN

Code: 7F606

Prerequisite : Nil

L	T	P	C
2	1	0	3

Course Objectives: Learn principles of Finite state machine, finite automation models, and transition diagrams. Understand regular languages and expressions for writing grammars. Understand context free grammars useful in designing compilers. Study the design and working of a complier .Study the role of grammars in compiler design.Learn a various parsing techniques for design of compilers.

Course Outcomes:

At the end of this course the student will be able to

1. Discuss principles of Finite state machine, finite automation models, and transition diagrams. Design NFA , DFA and FSM transition with suitable examples expressions which are useful in text editors.
2. Describe regular languages, regular expressions , grammars and derivations of strings with suitable examples. Describe context free grammars, syntax analysis useful in designing compilers.
3. Design of PDA and Turing Machine.
4. Explain Overview of compiler its Environment phases and features of Lexical Analyzer, LEX tool Describe Top down parsing technique, Recursive decent parsing with back tracking, Ambiguous grammar, Predictive parsing, LL(1).
5. Demonstrate and solve problems on SLR, CLR, LALR, operator precedence parser, LR (O), LR(1), LR(K) grammar and use YACC tool.
6. Describe and use Semantic Analysis concepts to design compiler : and describe Intermediate code generation such as 3-address code form.

UNIT-I: Strings, Alphabet, Language, Operations, finite automaton model, acceptance of strings, and languages, deterministic finite automaton and non deterministic finite automaton, Equivalence between NFA to DFA conversion.

UNIT-II: Regular Languages, Regular sets, regular expressions, Constructing finite Automata for a given regular expressions, Conversion of Finite Automata to Regular expressions. closure properties of regular sets (proofs not required).

Context Free Grammars: Context free grammar, derivation trees, Right most and leftmost derivation of strings. Ambiguity in context free grammars. Minimization of Context Free Grammars. Chomsky normal form, Greiback normal form,

UNIT-III Push down automata: definition, model, acceptance of CFL, Introduction to DCFL and DPDA.

Turing Machine : Turing Machine, definition, model, design of TM, recursively enumerable languages. Chomsky hierarchy of languages

UNIT VI: Overview of compiler – Environment, pass, phase, phases of compiler, LEX tool, Top Down Parsing: Top down parsing technique, Recursive decent parsing with back tracking, Ambiguous grammar, Elimination of left recursion, Left factoring, Predictive parsing, LL(1).

UNIT V Bottom up parsing: shift reduce parser SLR, CLR, LALR, operator precedence parser, LR (O), LR(1), LR(K) grammar, YACC tool.

UNIT VI: Semantic Analysis: Syntax directed translation, S- Attributed, L Attributed definition, Type checker, Intermediate code generation: 3-address code form, DAG. Code optimization: Optimization, loop optimization, peep-hole optimization, Symbol table format

TEXTBOOKS:

1. Introduction to Automata Theory Languages and Computation. Hopcroft H.E. and Ullman J. D. Pearson Education
2. Introduction to Theory of Computation? Sipser 2nd edition Thomson
3. Compilers Principles, Techniques and Tools Aho, Ullman, Ravisethi, Pearson Education

REFERENCES:

1. Introduction to Computer Theory, Daniel I.A. Cohen, John Wiley.
2. Introduction to languages and the Theory of Computation ,John C Martin, TMH
3. Elements of Theory of Computation?, Lewis H.P. & Papadimition C.H. Pearson /PHI.
4. Theory of Computer Science Automata languages and computation -Mishra and Chandrashekaran, 2nd edition, PHI Course Requirements.
5. Modern Compiler Construction in C , Andrew W.Appel Cambridge University Press.
6. Compiler Construction, LOUDEN, Thomson